Séance 3 — Déploiement managé

Master 2 · YNOV Campus Montpellier · Industrialisation de l'IA dans le Cloud · 2026-2027 Plateforme : Microsoft Azure · Énoncé apprenant

En-tête de séance

Élément	Détail
Durée	3 h 30 de TP
Modalité	Individuel, poste personnel, compte Azure for Students
Marche ajoutée	Le déploiement managé : exécuter dans le cloud un conteneur déjà construit, le configurer, l'observer, le charger
Prérequis consommé	L'image de l'API d'inférence produite en séance 2, publiée sur le registre ACR créé en séance 1
Hors périmètre	Aucun nouveau modèle, aucun réseau de neurones (séance 4), aucune optimisation d'inférence — quantization, ONNX (séance 5), aucun Kubernetes (projet fil rouge)
Livrable	Un dossier `s3-<votre-nom>/` (voir section « Livrable attendu »)

Prérequis

Vérifiez ces six points **avant** le début de la séance. Un prérequis manquant coûte 30 minutes de TP, et cette séance n'a pas 30 minutes de marge.

1. Un compte **Azure for Students** actif, avec du crédit restant. Contrôle : `az account show` répond.
2. **Azure CLI** en version 2.60 ou supérieure (`az version`). Les commandes de conteneur ont changé de nom sur les versions antérieures.
3. **Docker Desktop** installé et démarré (`docker info` répond).
4. **Python 3.10+** avec un environnement virtuel utilisable, pour installer Locust.
5. **L'artefact de la séance 2** : votre image d'API d'inférence, présente dans votre registre ACR. Contrôle : `az acr repository list --name "<votre-registre>" --output table` doit lister le dépôt de votre API d'inférence, et pas seulement la sonde de santé de S1.
6. **Votre `charge.json` de la séance 2**, la ligne de lésion réelle portant les quarante variables attendues par votre modèle. Vous l'avez produite en S2 : récupérez-la dans votre dossier de travail de S2. Sans elle, vous ne pourrez pas appeler `/predict` aujourd'hui — l'API refuse toute charge incomplète, et personne ne retape quarante champs à la main. Contrôle : `python3 -c "import json;print(len(json.load(open('charge.json'))))"` doit afficher le nombre de colonnes annoncé par votre `/model/info`.

Si ce dernier contrôle échoue, ne commencez pas la séance : rattrapez d'abord la publication de S2, ou utilisez l'image de secours décrite en annexe D. Vous perdrez alors les points liés à votre propre artefact, mais pas ceux de la séance.

Contrat d'interface attendu de votre image S2

Toutes les commandes du TP supposent que votre conteneur respecte ce contrat. Vérifiez-le maintenant, en local.

Élément	Attendu
Port d'écoute	Lu depuis la variable d'environnement `PORT`, valeur par défaut `8000`
Interface d'écoute	`0.0.0.0`, jamais `127.0.0.1`
`GET /health`	`200` et un JSON contenant au minimum `status` et `model_loaded`
`GET /model/info`	`200` et un JSON contenant la version du modèle
`POST /predict`	`200` sur une charge utile clinique complète, `400` sur une charge incomplète — la validation précède la prédiction
Réponse de `/predict`	Au minimum `probabilite_maligne`, `seuil` et `decision` : une probabilité sans son seuil n'est pas une décision
Architecture de l'image	`linux/amd64`
Serveur	Gunicorn ou équivalent de production, pas le serveur de développement de Flask

Si votre image ne respecte pas le point « port lu depuis `PORT` », corrigez-le et republiez avant la séance. C'est le point qui fait échouer le déploiement de la partie 2 dans sept cas sur dix.

Objectifs

À la fin de la séance, vous savez :

- tester la faisabilité d'un service Azure **avant** d'investir deux heures dessus, et basculer sur une cible de repli sans perdre l'objectif pédagogique ;
- déployer sur un service managé un conteneur tiré depuis un registre privé, sans jamais recopier de mot de passe ;
- accorder à ce service le droit de lire votre registre, par une identité et non par un secret partagé ;
- séparer l'artefact de sa configuration : variables d'environnement, secrets, port, sonde de santé ;
- lire les logs d'un conteneur distant et diagnostiquer un démarrage qui échoue à partir du message, pas à partir d'une intuition ;
- conduire un test de charge progressif, relever p50, p95, p99 et le taux d'erreur, et identifier un point de rupture **mesuré** ;
- décider ce qu'une inférence doit laisser comme trace, ce qu'elle ne doit jamais laisser, et pendant combien de temps ;
- détruire vos ressources.

Contexte métier — le fil rouge du module

Chez **DermaScan**, vous avez construit deux choses. En séance 1, vous avez appris à emballer une application dans une image et à la déposer dans un registre privé. En séance 2, vous avez entraîné sur le tabulaire ISIC 2024 un classifieur de lésions cutanées — une maligne pour mille vingt bénignes —, vous

avez choisi un **seuil de décision** adossé à un rappel visé, et vous avez emballé le tout dans une API d'inférence conteneurisée qui ne tourne que sur votre poste.

Ce matin, la directrice médicale vous pose la question qui change tout : « les dermatologues de trois centres de dépistage veulent interroger votre modèle depuis leur poste de consultation, cet après-midi. Où est l'URL ? »

Il n'y a pas d'URL. Une image dans un registre n'est pas un service : c'est un artefact au repos. **Aujourd'hui, vous n'entraînez rien.** Vous prenez ce que vous avez déjà et vous le mettez en ligne, joignable publiquement, configuré, observable. Puis vous répondez à la question suivante, qu'on ne vous posera pas mais qui vous sera opposée à la première panne : combien de praticiens simultanés ce service supporte-t-il avant de se dégrader ?

Et parce qu'un service en ligne produit des traces, vous terminerez par la décision qui engage juridiquement DermaScan : que journalise-t-on d'une inférence, et pendant combien de temps. Cette fois, la question n'est pas théorique — **vous servez des données concernant la santé**, et la trace d'une inférence porte un patient.

Convention de lecture de l'énoncé

Deux notations, à ne pas confondre :

Notation	Signification	Quand la remplir
`_____`	**Valeur à décider par vous** avant d'exécuter la commande	Avant l'exécution
Cellule vide d'un tableau de mesures	**Valeur à relever** dans la sortie d'une commande	Après l'exécution

L'énoncé n'affirme **aucune latence, aucun débit, aucun point de rupture**. Ces valeurs dépendent du plan de service que vous aurez choisi, de la région, du débit du réseau de la salle et de la machine depuis laquelle vous lancez Locust. Deux apprenants n'obtiendront pas les mêmes chiffres, et un chiffre recopié sur le voisin est immédiatement détectable : il ne correspondra pas à votre propre courbe.

Tableau de décisions

Complétez ce tableau **maintenant**, avant la partie 1. Toutes les commandes du TP s'y réfèrent.

Contraintes de nommage à respecter :

- le nom de l'application web est **globalement unique sur Azure** : il devient le sous-domaine `<nom>.azurewebsites.net`, en minuscules, chiffres et tirets ;
- reprenez **exactement** le suffixe personnel de la séance 1, sans quoi vous perdrez la traçabilité entre vos trois séances ;
- le registre ACR est celui de la séance 1 : s'il a été détruit au nettoyage de S1, recréez-le et republiez votre image avant de commencer.

Décision	Contrainte	Votre valeur
Suffixe personnel	identique à S1	`_____`

Région Azure	une seule, la même partout, choisie en partie 1	`_____`
Nom du groupe de ressources	préfixé `rg-dermascan-s3-`	`_____`
Nom du registre ACR existant	celui de S1	`_____`
Dépôt et tag de l'image S2	ce qui est réellement dans le registre	`_____`
Nom du plan de service	préfixé `plan-dermascan-`	`_____`
Nom de l'application web	unique sur Azure, minuscules et tirets	`_____`

Question 0.1 — Le nom de l'application web est unique sur tout Azure, exactement comme l'était le nom du registre ACR en séance 1. Cette fois, une conséquence supplémentaire apparaît, que le registre n'avait pas : laquelle, du point de vue d'un attaquant qui cherche des services mal protégés ? Deux lignes dans `FAISABILITE.md`.

Partie 1 — Vérification de faisabilité et choix de la cible

Cette partie est la plus importante de la séance, et c'est la moins spectaculaire.

Pourquoi elle existe. En 2025-2026, la promotion précédente a perdu une demi-journée à déboguer un déploiement qui ne pouvait pas fonctionner : les abonnements Azure for Students refusaient la création d'App Service dans toutes les régions testées, par une politique d'abonnement qu'aucun message d'erreur ne nommait clairement. La compétence professionnelle à acquérir ici n'est pas « savoir déployer » : c'est **savoir en combien de temps on saura qu'on ne pourra pas déployer**, et avoir décidé à l'avance ce qu'on fera alors.

Règle de la séance : vous vous donnez 20 minutes. Passé ce délai, vous basculez sur la cible de repli de l'annexe B, sans délibérer et sans appeler le formateur pour négocier. Le reste du TP est identique sur toutes les cibles.

Créez `FAISABILITE.md` et reportez-y tous les relevés de cette partie.

1.1 — Authentification et variables de séance

Authentifiez-vous avec `az login` — ou `az login --use-device-code` sur un compte scolaire avec MFA —, fixez explicitement l'abonnement avec `az account set --subscription "_____"`, puis contrôlez l'identité réellement active avec `az account show --output table`.

Déclarez ensuite vos décisions sous forme de variables de shell. Vous reprendrez ce bloc à chaque nouveau terminal ouvert : les `export` ne survivent pas à la fermeture d'un shell. Sous PowerShell, la syntaxe est `$env:NOM = "valeur"`.

```
export SUFFIX="_____"
export RG="rg-dermascan-s3-$SUFFIX"
export ACR_NAME="_____"
export IMAGE_REPO="_____"
export IMAGE_TAG="_____"
echo "$SUFFIX | $RG | $ACR_NAME | $IMAGE_REPO | $IMAGE_TAG"   # aucune vide
```

1.2 — Sonder les contraintes de l'abonnement

Trois contraintes peuvent bloquer un déploiement, et elles ne produisent pas le même message d'erreur. Sondez-les dans cet ordre : de la plus silencieuse à la plus bruyante.

Contrainte 1 — la politique de régions autorisées. Une politique d'abonnement peut restreindre les régions de déploiement. Elle est invisible tant qu'on ne la déclenche pas.

Contrainte 2 — les fournisseurs de ressources enregistrés. Un abonnement neuf n'a pas tous les fournisseurs activés. Un fournisseur non enregistré produit une erreur `MissingSubscriptionRegistration` que beaucoup interprètent à tort comme un manque de droits. Les trois fournisseurs dont dépend la séance doivent tous répondre `Registered` ; un enregistrement manquant se rattrape de façon asynchrone, en une à trois minutes.

```
# Sonde 1 : une sortie vide signifie qu'aucune politique de région n'est active.
az policy assignment list --output tsv \
  --query "[?displayName=='Allowed resource deployment
regions'].parameters.listOfAllowedLocations.value[]"
# Sonde 2 : état d'enregistrement des trois fournisseurs.
for ns in Microsoft.Web Microsoft.ContainerInstance Microsoft.ContainerRegistry; do
  az provider show --namespace "$ns" --query registrationState --output tsv
done
az provider register --namespace "_______" --wait
```

Contrainte 3 — le quota réel. Un fournisseur enregistré dans une région autorisée peut encore refuser la création, pour cause de quota nul sur le type de ressource. C'est exactement ce qui s'est produit en 25-26. Seule une tentative réelle le révèle.

C'est un test de faisabilité par tentative réelle et non par simulation : seule la création effective d'un plan Linux révèle un quota nul. En cas d'échec, lisez le code d'erreur **avant** de changer quoi que ce soit. En cas de succès, le plan doit exister et porter l'état `Ready`.

```
export LOCATION="_______"
export PLAN_NAME="_______"
az group create --name "$RG" --location "$LOCATION" \
  --tags projet=dermascan seance=s3 proprietaire="_______" a_detruire="_______"
az appservice plan create --name "$PLAN_NAME" --resource-group "$RG" \
  --location "$LOCATION" --is-linux --sku "_______"
az appservice plan show --name "$PLAN_NAME" --resource-group "$RG" \
  --query "{nom:name, sku:sku.name, etat:properties.status}" --output table
```

Les trois sondes échouent de trois façons différentes. Ce tableau vous évite de confondre un droit manquant avec un quota nul :

Sonde	Ce qu'un échec produit	Interprétation
Politique de régions	`RequestDisallowedByPolicy`	La région est interdite, pas le service
Fournisseur	`MissingSubscriptionRegistration`	Rien à voir avec vos droits : le fournisseur n'est pas activé
Création du plan	message de quota, ou création silencieusement refusée	Quota nul : aucune commande ne le contournera

Mesure 1 — sondes de faisabilité de l'abonnement : politiques de région, fournisseurs, création d'un plan Linux. Complétez ce relevé dans `FAISABILITE.md` :

Sonde	Commande source	Votre relevé
Régions autorisées par la politique	`az policy assignment list`	
`Microsoft.Web`	`az provider show`	
`Microsoft.ContainerInstance`	`az provider show`	
`Microsoft.ContainerRegistry`	`az provider show`	

Création du plan App Service Linux	`az appservice plan create`	succès / échec
Code d'erreur exact, le cas échéant	sortie de la commande	
Durée totale de la sonde	chronomètre	

1.3 — Choix de la cible et engagement

Arbitrez maintenant, à partir de **votre** relevé et non de celui du voisin.

Cible	Condition d'éligibilité	Où est décrite la suite
A — App Service for Containers	Le plan Linux a été créé, état `Ready`	Parties 2 à 6 de l'énoncé principal
B — Azure Container Instances	Le plan a échoué, mais `Microsoft.ContainerInstance` est `Registered`	Annexe B, section B.1, puis parties 3 à 6 adaptées
C — Exécution locale instrumentée	Les deux précédentes ont échoué, ou vous avez dépassé les 20 minutes	Annexe B, section B.3, puis parties 3 à 6 adaptées

Mesure 2 — cible retenue, fait déclencheur et heure de la décision. Inscrivez dans `FAISABILITE.md` :

Élément de la décision	Votre réponse
Cible retenue	
Fait déclencheur, cité avec le message d'erreur s'il y en a un	
Heure de la décision	
Ce que cette cible ne permettra pas de montrer aujourd'hui	

Question 1.1 — Les trois cibles diffèrent par le niveau d'abstraction. Pour chacune, indiquez ce que vous n'avez **pas** à gérer, en reprenant la grille IaaS / PaaS / SaaS de la séance 1.

Cible	Modèle de service	Ce que vous ne gérez pas	Ce qui reste à votre charge
App Service for Containers	`_____`		
Azure Container Instances	`_____`		
Docker sur votre poste	`_____`		

Question 1.2 — Relevez dans la documentation tarifaire Azure le coût mensuel annoncé du SKU que vous venez de choisir, dans votre région. Rapportez-le au crédit restant de votre compte, relevé dans `Cost Management`. Combien de jours de fonctionnement continu votre crédit finance-t-il ? Ce chiffre justifie à lui seul le nettoyage de fin de séance.

Partie 2 — Déploiement du conteneur depuis ACR

Vous avez un plan de service, c'est-à-dire de la capacité de calcul réservée. Vous n'avez pas encore d'application. Cette partie installe l'application dessus et, surtout, règle la question qui bloque la moitié des déploiements réels : **comment le service prouve-t-il au registre qu'il a le droit de tirer l'image ?**

2.1 — Retrouver l'image exacte à déployer

On ne déploie pas « la dernière image ». On déploie une image nommée, dont on a relevé le digest.

Reconstruisez le serveur de connexion depuis Azure et non de mémoire, vérifiez que le tag existe réellement dans le registre, puis lisez le manifeste : il porte le digest, l'architecture, l'OS et la date de publication. L'architecture **doit** être `amd64` ; si elle est `arm64`, arrêtez ici et republiez avec `--platform linux/amd64`.

```
export LOGIN_SERVER=$(az acr show --name "$ACR_NAME" --query loginServer --output tsv)
export IMAGE_FULL="$LOGIN_SERVER/$IMAGE_REPO:$IMAGE_TAG"
az acr repository show-tags --name "$ACR_NAME" --repository "$IMAGE_REPO" --output table
az acr manifest list-metadata --registry "$ACR_NAME" --name "$IMAGE_REPO" --output table
az acr manifest list-metadata --registry "$ACR_NAME" --name "$IMAGE_REPO" \
  --query "[?tags[0]=='$IMAGE_TAG'].digest" --output tsv   # digest du tag
```

La table n'affiche pas de colonne de tag. Si l'image a été construite avec `--provenance=false`, comme en séance 2, elle ne compte qu'une ligne : l'image `amd64` elle-même. Sans cette option, Docker Desktop pousse **trois** manifestes : l'image `amd64`, une attestation de construction et un index qui les référence, ces deux derniers avec une architecture vide. Dans les deux cas, le digest du tag est celui qu'affiche `docker push` et que donne la dernière commande (celui de l'index s'il y en a un) ; l'architecture se lit sur la ligne qui en porte une.

Mesure 3 — identification exacte de l'image à déployer : nom complet, digest et architecture.

Relevé	Votre valeur
`loginServer`	
Nom complet de l'image (`IMAGE_FULL`)	
Digest `sha256` du tag déployé	
Architecture déclarée dans le manifeste	
Date de publication de ce tag	

Question 2.1 — Vous déployez un tag, mais vous relevez un digest. Que se passerait-il si, demain, un camarade poussait une autre image sous le **même** tag dans ce registre, et que votre service redémarrait ? Le digest relevé aujourd'hui vous servirait-il à quelque chose ? Répondez dans `DEPLOIEMENT.md`.

2.2 — Créer l'application web à partir de l'image

L'unicité globale du nom se vérifie avant de tenter la création. La création se fait sur le plan existant, en pointant l'image du registre. À cet instant, l'application n'a **aucun** droit de lecture sur ce registre : le premier pull va échouer, et c'est attendu.

Un réglage se pose dès la création, parce que sans lui le conteneur ne démarrera jamais, même une fois le droit de pull accordé : Azure ne devine pas sur quel port votre conteneur écoute. Il tente le port 80 (puis 8080), puis abandonne. Le réglage `WEBSITES_PORT` lui dit où frapper. Les autres réglages attendront la partie 3.

```
export APP_NAME="_______"
az webapp list --query "[?name=='$APP_NAME']" --output table
az webapp create --resource-group "$RG" --plan "$PLAN_NAME" \
  --name "$APP_NAME" --container-image-name "$IMAGE_FULL"
az webapp config appsettings set --resource-group "$RG" --name "$APP_NAME" \
  --settings WEBSITES_PORT="_______" > /dev/null
export APP_URL="https://$APP_NAME.azurewebsites.net"
```

Observation immédiate, avant toute correction. Interrogez l'URL maintenant avec `curl -s -o /dev/null -w "code=%{http_code} duree=%{time_total}s\n" "$APP_URL/health"` : `-o /dev/null` jette le corps de la réponse, `-w` n'affiche que le code HTTP et la durée totale.

Mesure 4 — comportement du service avant tout octroi de droit sur le registre.

Relevé avant octroi des droits	Votre valeur
Code HTTP retourné	
Durée de la réponse	
Contenu de la page renvoyée (une ligne de résumé)	

Ne corrigez rien encore. La partie 4 vous demandera de retrouver, dans les logs, la trace exacte de cet échec.

2.3 — Identité d'accès au registre

Il existe deux façons de donner à l'application le droit de tirer l'image. Vous allez implémenter la bonne, et savoir nommer l'autre.

Méthode à ne pas utiliser, mais à savoir nommer. Activer l'utilisateur administrateur du registre (`az acr update --admin-enabled true`) et coller son mot de passe dans la configuration de l'application. C'est la méthode que vous trouverez dans la plupart des tutoriels. Vous avez déjà listé ses défauts en séance 1, partie 4.2.

Méthode retenue — l'identité managée. L'application reçoit une identité propre dans Entra ID, à laquelle on accorde un rôle sur le registre. Aucun mot de passe n'existe, donc aucun mot de passe ne fuit.

La logique de la séquence. On crée d'abord l'identité, on relève ensuite son identifiant de principal, puis l'identifiant du registre qui servira de **portée** au droit, on attribue le rôle, on demande à App Service d'utiliser l'identité plutôt que des identifiants, et on redémarre pour forcer un nouveau pull. Chaque

commande `az webapp` prend `--resource-group "$RG" --name "$APP_NAME"`.

Geste	Commande	Ce qu'il établit
1	`az webapp identity assign`	Une identité affectée par le système : elle naît et meurt avec l'application, il n'y a rien à révoquer manuellement
2	`az webapp identity show --query principalId --output tsv`	`PRINCIPAL_ID`, le principal à qui le rôle sera accordé
3	`az acr show --name "$ACR_NAME" --query id --output tsv`	`ACR_ID`, la portée du droit : le registre et non l'abonnement, soit le moindre privilège à la lettre
4	`az role assignment create --assignee-object-id "$PRINCIPAL_ID" --assignee-principal-type ServicePrincipal --scope "$ACR_ID" --role "_______"`	Le rôle **minimal** permettant de tirer une image : à vous de le nommer
5	`az webapp config set --generic-configurations '{"acrUseManagedIdentityCreds": true}'`	L'usage de l'identité à la place d'un couple identifiant / mot de passe
6	`az webapp restart`, puis `az role assignment list --scope "$ACR_ID"`	Un nouveau pull, et le contrôle de l'attribution effective. En table, la colonne `Principal` affiche l'identifiant d'**application** de l'identité, pas le `principalId` du geste 2 : ne concluez pas que l'attribution n'est pas la vôtre

Mesure 5 — identité managée, rôle accordé, portée du rôle et délai de propagation.

Relevé	Votre valeur
`principalId` de l'identité managée	
Rôle accordé	
Portée exacte du rôle (chemin complet)	
Délai entre l'attribution du rôle et le premier pull réussi	

Question 2.2 — Il existe un rôle plus large qui aurait aussi fonctionné, et un rôle encore plus large que certains tutoriels recommandent par facilité. Nommez-les tous les deux, et dites précisément ce que votre application pourrait faire en plus avec chacun. Trois lignes dans `DEPLOIEMENT.md`.

Question 2.3 — L'attribution d'un rôle n'est pas instantanée : elle doit se propager. Si votre premier redémarrage échoue encore, attendez et relancez avant de conclure à une erreur de configuration. Combien de temps avez-vous dû attendre ? Notez-le : c'est une donnée utile pour écrire un script de déploiement automatisé.

2.4 — Premier démarrage et mesure du démarrage à froid

Sondez `/health` toutes les dix secondes en horodatant chaque essai, et interrompez par `Ctrl+C` dès le premier `200`. `-m 30` borne chaque essai à 30 secondes : sans lui, une requête peut rester suspendue plus d'une minute. Notez l'heure de départ **avant** de lancer la boucle.

```
for i in $(seq 1 30); do
  printf "%s " "$(date +%H:%M:%S)"
  curl -s -m 30 -o /dev/null -w "%{http_code}\n" "$APP_URL/health"
  sleep 10
done
```

Une fois le `200` obtenu, relevez le corps complet de `/health`, puis lancez une prédiction réelle. La charge utile est le `charge.json` de votre séance 2 : une ligne de lésion réelle, toutes colonnes attendues présentes. `-d @fichier` envoie le contenu du fichier comme corps de requête — c'est ce qui vous dispense de retaper quarante champs.

Si vous ne retrouvez pas votre `charge.json`, reconstruisez-le depuis votre paquet sérialisé et le fichier d'annotation de S2 plutôt que de l'écrire à la main : `/model/info` vous donne la liste exacte des colonnes attendues, dans l'ordre.

```
curl -s "$APP_URL/health"
curl -s "$APP_URL/model/info" | python3 -m json.tool | grep -v '^        "'   # masque la liste des colonnes
curl -s -X POST "$APP_URL/predict" -H "Content-Type: application/json" -d @charge.json
```

Pour chronométrer, rejouez la même prédiction cinq fois en n'affichant que sa durée : `curl -s -o /dev/null -w "%{time_total}s\n" -X POST "$APP_URL/predict" -H "Content-Type: application/json" -d @charge.json`.

Mesure 6 — premier démarrage réussi : délai jusqu'au premier `200`, état du modèle, latence de la première et de la cinquième prédiction.

Mesure du premier démarrage	Votre valeur
Durée entre le `restart` et le premier `200` sur `/health`	
Codes HTTP observés pendant l'attente, dans l'ordre (`000` : délai client de 30 s dépassé)	
Champ `model_loaded` de la réponse `/health`	
Version de modèle retournée par `/model/info`	
`probabilite_maligne` et `decision` retournés par `/predict`	
Le `seuil` appliqué par le service est-il celui de votre paquet S2 ?	
Durée de la **première** requête `/predict`	
Durée de la **cinquième** requête `/predict` identique	

Question 2.4 — L'écart entre la première et la cinquième requête est-il nul, faible ou important ? Proposez **deux** causes possibles, et dites pour chacune comment vous pourriez la confirmer ou l'écarter avec les outils dont vous disposez aujourd'hui. C'est une question d'hypothèses, pas de connaissance : une réponse fausse mais testable vaut mieux qu'une réponse juste récitée.

Partie 3 — Configuration d'exécution

Votre service tourne. Il tourne avec la configuration par défaut de l'image, c'est-à-dire la configuration de développement de la séance 2. Un artefact immuable plus une configuration externe : c'est la promesse que vous avez mesurée en séance 1 avec le conteneur `bis`. Vous allez l'exercer pour de bon.

3.1 — Variables d'environnement et port

Vous avez posé `WEBSITES_PORT` en 2.2 : il dit à la plateforme où frapper. Il reste à dire à votre code, explicitement, où écouter — `PORT` —, et à lui donner le reste de sa configuration.

Ces réglages deviennent des variables d'environnement dans le conteneur, sans reconstruction d'image. Relevez ensuite la configuration effective, et notamment son nombre d'entrées.

```
az webapp config appsettings set --resource-group "$RG" --name "$APP_NAME" \
  --settings PORT="_______" MODEL_VERSION="_______" \
    LOG_LEVEL="_______" ENVIRONMENT="_______" SEUIL="_______"
az webapp config appsettings list --resource-group "$RG" --name "$APP_NAME" --output table
```

Le réglage `SEUIL` n'est pas un réglage de plus. Votre API de séance 2 lit son seuil de décision dans le paquet sérialisé, mais accepte qu'une variable d'environnement le remplace. Donnez-lui ici **exactement** la valeur que votre entraînement avait retenue, relevée dans votre `MESURES.md` de S2. Reportez-la aussi dans votre rendu : un seuil appliqué en production sans trace écrite est indéfendable. Vous l'éprouverez en 3.3.

Question 3.1 — `WEBSITES_PORT` et `PORT` doivent-ils porter la même valeur ? Lequel des deux est lu par la plateforme, lequel est lu par votre code ? Que se passe-t-il s'ils diffèrent ? Vous vérifierez expérimentalement en partie 4 — répondez d'abord, mesurez ensuite.

3.2 — Secrets

Vous allez avoir besoin, en partie 6, d'un sel cryptographique pour pseudonymiser les identifiants de patient dans les journaux. Ce sel est un **secret** : s'il fuite, la pseudonymisation est réversible par force brute et ne protège plus rien — et ce qu'elle protège ici, ce sont des données concernant la santé.

Le sel est engendré localement et poussé comme réglage d'application. Il ne doit apparaître dans aucun fichier de votre rendu, ni dans un historique de commandes partagé. Au contrôle, observez attentivement **comment** la valeur est affichée.

```
export INFERENCE_SALT=$(openssl rand -hex 16)
az webapp config appsettings set --resource-group "$RG" --name "$APP_NAME" \
  --settings INFERENCE_LOG_SALT="$INFERENCE_SALT" > /dev/null
az webapp config appsettings list --resource-group "$RG" --name "$APP_NAME" \
  --query "[?name=='INFERENCE_LOG_SALT']" --output table
```

Mesure 7 — visibilité réelle du secret `INFERENCE_LOG_SALT` à ses trois points d'exposition.

Contrôle	Votre observation

La valeur du sel est-elle masquée dans la sortie du CLI ?	
Est-elle visible dans le portail Azure, onglet `Configuration` ?	
Est-elle lisible par un processus qui s'exécuterait dans le conteneur ?	

Question 3.2 — Un réglage d'application n'est pas un coffre-fort : c'est un moyen de transport. Nommez le service Azure qui joue le rôle de coffre-fort, et décrivez en deux lignes ce que la combinaison « ce service + l'identité managée que vous avez créée en 2.3 » permettrait de supprimer complètement de votre configuration. Vous ne l'implémentez pas aujourd'hui ; vous devez savoir le nommer.

3.3 — Sonde de santé

Une sonde de santé n'est pas un endpoint de plus : c'est le contrat par lequel la plateforme décide de retirer une instance du service. Sans sonde déclarée, la plateforme route du trafic vers une instance morte jusqu'à ce qu'un humain s'en aperçoive.

Déclarez le chemin de la sonde : la plateforme l'interroge périodiquement et retire du routage toute instance qui échoue de façon répétée. Contrôlez la valeur enregistrée, puis redémarrez **une fois**, explicitement, et chronométrez le retour au service avec la boucle de 2.4 : c'est ce que coûte un `restart`.

```
az webapp update --resource-group "$RG" --name "$APP_NAME" \
  --set siteConfig.healthCheckPath="_______"
az webapp config show --resource-group "$RG" --name "$APP_NAME" \
  --query healthCheckPath --output tsv
az webapp restart --resource-group "$RG" --name "$APP_NAME"
```

Puis éprouvez le réglage `SEUIL`, et c'est le relevé le plus instructif de la partie. Abaissez-le à une valeur volontairement basse — le dixième de la vôtre, par exemple —, **sans redémarrer** : un `appsettings set` recycle déjà le conteneur. Rejouez **la même** `charge.json` jusqu'à voir le nouveau seuil dans la réponse, en notant le temps écoulé, relevez cette réponse, puis remettez votre valeur de S2, toujours sans redémarrer. Si rien ne bouge au bout de trois minutes, interrompez par `Ctrl+C` : votre API ne lit sans doute pas `SEUIL`, et ce constat est un relevé à part entière.

```
export SEUIL_BAS="_______"
az webapp config appsettings set --resource-group "$RG" --name "$APP_NAME" \
  --settings SEUIL="$SEUIL_BAS" > /dev/null
until curl -s -X POST "$APP_URL/predict" -H "Content-Type: application/json" -d
@charge.json \
  | python3 -c "import sys,json; sys.exit(abs(json.load(sys.stdin)['seuil'] - $SEUIL_BAS)
> 1e-9)" 2>/dev/null; do
  date +%H:%M:%S; sleep 5
done
curl -s -X POST "$APP_URL/predict" -H "Content-Type: application/json" -d @charge.json
```

Mesure 8 — effet de la configuration complète après redémarrage : sonde enregistrée, délai de retour au service, nombre de réglages, et effet du seuil à entrée et à modèle rigoureusement identiques, obtenu sans redémarrage.

Relevé après redémarrage	Votre valeur
Chemin de sonde enregistré	
Durée du redémarrage jusqu'au premier `200`	
Délai entre le `appsettings set` du seuil abaissé et l'apparition du nouveau seuil dans la réponse	
Nombre total de réglages d'application	
La valeur de `MODEL_VERSION` apparaît-elle dans la réponse de `/model/info` ?	
`probabilite_maligne`, avec votre seuil de S2	
`seuil` et `decision` retournés, avec votre seuil de S2	
`probabilite_maligne`, avec le seuil abaissé	
`seuil` et `decision` retournés, avec le seuil abaissé	

Question 3.3 — Votre `/health` retourne `model_loaded`. Imaginez deux versions de cette sonde :

- version 1 : elle retourne `200` dès que le processus Python répond ;
- version 2 : elle retourne `200` uniquement si le modèle est chargé **et** si une inférence de test sur une entrée figée donne le résultat attendu.

Laquelle déployeriez-vous, et quel risque introduit celle que vous écartez ? Traitez le cas d'une instance qui répond mais dont le modèle n'a pas pu être chargé. Quatre lignes dans `DEPLOIEMENT.md`.

Question 3.4 — Même image, même modèle, même charge utile : la probabilité n'a pas bougé d'un millième et la décision a changé. Répondez en quatre lignes dans `DEPLOIEMENT.md` :

1. Qu'est-ce qui, du modèle ou du service, a changé de comportement ?
2. Le service déployé rend-il « les mêmes décisions » dans les deux cas ? C'est une question de vocabulaire, et elle se pose en audit : un rapport qui annonce « le modèle a changé d'avis » décrit-il correctement ce qui s'est passé ?
3. Un collègue abaisse ce réglage en production sans vous le dire, pour faire remonter le rappel. Quelle trace vous permettrait de le découvrir après coup ? Reliez votre réponse à la grille de la partie 6.
4. Un seuil modifiable sans redéploiement est-il un progrès ou un risque ? Donnez votre position **et** l'argument adverse.

Partie 4 — Observabilité et diagnostic d'un démarrage qui échoue

Jusqu'ici, quand quelque chose ne marchait pas, vous aviez le conteneur sous la main. Il est maintenant à plusieurs centaines de kilomètres, vous n'avez pas de shell dessus, et la seule chose qui sorte de la boîte, ce sont les logs. Savoir les lire n'est pas une commodité : c'est la seule chose qui vous reste.

4.1 — Activer et lire les logs

Sans activation préalable de la journalisation, `log tail` affiche une sortie vide et vous concluriez à tort que l'application n'écrit rien. Une fois le flux ouvert, **laissez ce terminal ouvert** tout le reste de la séance.

```
az webapp log config --resource-group "$RG" --name "$APP_NAME" \
  --docker-container-logging filesystem --application-logging filesystem \
  --level "_______"
az webapp log tail --resource-group "$RG" --name "$APP_NAME"
```

Dans un **second terminal**, rejouez les variables de session, générez du trafic sur les trois endpoints — un `GET /health`, un `GET /model/info` et un `POST /predict` volontairement incomplet, par exemple `-d '{"age_approx": 55}'` — une seule variable sur les quarante attendues — et observez le premier terminal.

Mesure 9 — ce que le flux de logs de la plateforme laisse réellement voir, charge utile comprise.

Observation dans le flux de logs	Votre relevé
Le nom du conteneur apparaît-il en préfixe de chaque ligne ?	
Retrouve-t-on la ligne d'accès de Gunicorn pour chaque requête ?	
Que journalise votre API sur un `/predict` incomplet ?	
La liste des variables manquantes apparaît-elle dans les logs ?	
Délai approximatif entre la requête et l'apparition de sa ligne	
La charge utile complète du `POST` apparaît-elle dans les logs ?	

La dernière ligne du tableau est à retenir pour la partie 6. Si la réponse est « oui », vous avez déjà un problème de conformité en production.

Récupérez également l'archive complète des logs, qui fait partie du livrable, avec `az webapp log download --resource-group "$RG" --name "$APP_NAME" --log-file logs_s3.zip`.

4.2 — Provoquer et diagnostiquer un démarrage qui échoue

Vous allez **casser votre service volontairement**, puis le réparer en vous appuyant uniquement sur les logs. C'est un exercice, pas un accident : notez tout dans `JOURNAL_INCIDENT.md`, au format d'un vrai compte rendu d'incident.

Étape 1 — la panne. Faites croire à la plateforme que le conteneur écoute sur un port où personne n'écoute.

Donnez à `WEBSITES_PORT` une valeur volontairement fausse — un port sur lequel votre application n'écoute certainement pas — puis redémarrez.

```
az webapp config appsettings set --resource-group "$RG" --name "$APP_NAME" \
  --settings WEBSITES_PORT="_______" > /dev/null
az webapp restart --resource-group "$RG" --name "$APP_NAME"
```

Étape 2 — l'observation. Sans corriger, et sans consulter l'annexe de dépannage, comparez le comportement vu de l'extérieur — `curl -s -m 60 -o /dev/null -w "code=%{http_code} duree=%{time_total}s\n" "$APP_URL/health"`, à répéter toutes les trente secondes pendant dix minutes — et le comportement vu de l'intérieur, dans `az webapp log tail --resource-group "$RG" --name "$APP_NAME"`.

Mesure 10 — diagnostic de la panne provoquée : code vu du client, ligne de log fautive, nombre de tentatives.

Élément du diagnostic	Votre relevé
Code HTTP vu par le client	
Délai avant la réponse d'erreur	
Ligne de log exacte qui nomme la cause, recopiée intégralement	
Combien de fois la plateforme a-t-elle retenté avant d'abandonner ?	
Le processus applicatif a-t-il démarré, oui ou non ?	
Temps écoulé entre le redémarrage et la ligne fautive	

Étape 3 — la réparation. Remettez la bonne valeur **sans redémarrer**, et mesurez le retour au service avec la boucle de 2.4.

Mesure 11 — délai de retour au service après correction.

Élément	Votre relevé
Durée entre la correction et le premier `200`	
A-t-il fallu un redémarrage explicite, ou la plateforme a-t-elle repris seule ?	

Question 4.1 — Le code HTTP vu par le client est le même que celui que vous aviez relevé en partie 2.2, avant d'accorder le rôle sur le registre. Les deux causes n'ont pourtant rien à voir. Répondez à ces trois questions dans `JOURNAL_INCIDENT.md` :

1. Qu'est-ce qui distingue les deux cas **du point de vue du client** ? Si rien, dites-le.
2. Qu'est-ce qui les distingue **dans les logs** ? Citez pour chaque cas la ligne discriminante.
3. Quelle conclusion en tirez-vous sur la valeur d'un code HTTP comme élément de diagnostic ?

4.3 — Métriques de plateforme

Les logs disent ce qui s'est passé. Les métriques disent comment le service se porte.

Le module `monitor` travaille sur l'identifiant de ressource de l'application, pas sur son nom ; relevez-le d'abord, puis demandez les métriques de la dernière heure. Avec `--interval PT1M`, la table sortirait une ligne par minute et par métrique — près de deux cents lignes, sans aucun total. `--interval PT1H` agrège l'heure en un seul point par métrique : `Total` pour les compteurs, `Average` pondérée pour le temps de réponse. Ne faites pas vous-même la moyenne des lignes par minute : une minute froide de six requêtes pèserait autant que deux cents requêtes rapides.

```
export APP_ID=$(az webapp show --resource-group "$RG" --name "$APP_NAME" --query id --
output tsv)
az monitor metrics list --resource "$APP_ID" \
  --metric "Requests" "Http5xx" "HttpResponseTime" --interval PT1H --output table
```

Le processeur et la mémoire ne sont pas des métriques de l'application mais du **plan** qui l'héberge. Notez cette seconde commande : vous la rejouerez juste après les paliers de la partie 5, pour la mesure 15.

```
export PLAN_ID=$(az appservice plan show --name "$PLAN_NAME" --resource-group "$RG" --
query id --output tsv)
az monitor metrics list --resource "$PLAN_ID" \
  --metric "CpuPercentage" "MemoryPercentage" --interval PT1M --output table
```

Mesure 12 — métriques agrégées de la plateforme sur l'heure écoulée.

Métrique	Votre relevé sur l'heure écoulée
Nombre total de requêtes	
Nombre de réponses `5xx`	
Temps de réponse moyen	

Question 4.2 — Ces métriques suffisent-elles à répondre à « mon service est-il lent pour mes utilisateurs ? » Justifiez en une phrase, en vous appuyant sur ce que vous allez mesurer en partie 5. La réponse tient en un mot, et ce mot n'est pas « moyenne ».

Partie 5 — Test de charge avec Locust

Un service qui répond à votre `curl` répond à **un** utilisateur. DermaScan en a trois centres. Vous allez mesurer, et non estimer, le nombre d'utilisateurs simultanés que votre déploiement supporte avant de se dégrader.

Règle d'honnêteté de la mesure. Locust s'exécute sur votre poste et traverse le réseau de l'établissement. Une partie de la latence que vous mesurerez n'appartient pas à votre service. Vous devrez l'estimer et le dire dans votre rapport ; un rapport qui attribue tout au serveur est un rapport faux.

5.1 — Installer Locust et écrire le scénario

Créez un environnement virtuel dédié, pour ne pas polluer celui de la séance 2 : `python3 -m venv .venv-locust`, `source .venv-locust/bin/activate`, `pip install locust`, puis contrôlez avec `locust --version`.

Créez ensuite `locustfile.py`. Vous l'écrivez vous-même, à partir de la logique et de la liste ci-dessous.

La logique. Un fichier Locust décrit un **utilisateur type**, pas un scénario global : une classe qui hérite de `HttpUser`, un temps de réflexion entre deux actions, et une méthode décorée par `@task` pour chaque geste que cet utilisateur peut faire. Locust en lance ensuite autant d'exemplaires que vous en demandez. Deux précautions conditionnent la validité de la mesure : chaque appel envoie une charge utile **variable**, sinon un cache éventuel côté serveur servirait toujours la même réponse ; et chaque tâche porte un `name` fixe, sinon les statistiques se dispersent sur autant de lignes que d'URL distinctes.

Les éléments attendus.

Élément	Contenu	Contrainte ou décision
Fonction de charge utile	Votre `charge.json` chargé **une fois** au démarrage du module, puis **perturbé** à chaque appel sur quelques variables numériques — `age_approx`, `clin_size_long_diam_mm`, les métriques `tbp_lv_*` de votre choix	Reprenez **vos** plages relevées en séance 2. Recopier quarante champs dans le code est une faute de conception : chargez le fichier
`wait_time`	`between(min, max)`, en secondes	Vos deux valeurs : un temps nul simulerait un robot, pas un technicien
Tâche `/predict`	`POST` en `catch_response=True` : succès si le code est `200`, échec qualifié sinon	Sans ce verdict explicite, une réponse `400` serait comptée comme un succès
Tâches `/health` et `/model/info`	Deux `GET` simples	Nommées `GET /health` et `GET /model/info`
Poids des trois tâches	Trois entiers passés à `@task(n)`	Vos décisions, justifiées par l'usage réel

Décision	Contrainte	Votre valeur

Temps de réflexion, borne basse et borne haute	en secondes, non nulles	`_____`
Poids de `/predict`, `/health` et `/model/info`	trois entiers, et non trois valeurs égales	`_____`
Variables perturbées à chaque appel, et amplitude de la perturbation	au moins deux variables ; l'amplitude reste dans les plages de S2	`_____`

Extrait : la tâche de prédiction, seul endroit où le verdict de réussite est explicite.

```
@task(_____)
def predire(self):
    with self.client.post("/predict", json=charge_utile_perturbee(),
                          name="POST /predict", catch_response=True) as reponse:
        if reponse.status_code == 200:
            reponse.success()
        else:
            reponse.failure(f"statut {reponse.status_code}")
```

Piège à éviter, et il coûte la mesure. Un `400` renvoyé par votre API est un échec *de votre scénario*, pas de votre service : si votre perturbation sort une variable de son domaine — un `age_approx` négatif, une catégorie inconnue —, vous mesurerez un taux d'échec qui ne dit rien de la charge. Contrôlez d'abord qu'un appel unique de votre fonction de charge utile passe en `200`, **avant** de lancer le premier palier.

Question 5.1 — Justifiez vos trois poids en une phrase chacun, à partir de l'usage réel décrit dans le contexte métier : des dermatologues qui soumettent une lésion depuis leur poste de consultation. Un poids identique sur les trois tâches produit une mesure qui ne représente aucun usage réel.

5.2 — Montée en charge progressive

Quatre paliers, exécutés dans l'ordre, sans interface graphique pour que la mesure soit reproductible. Chaque palier dure 3 minutes, dont la première sert à la stabilisation et n'est pas représentative.

`--headless` supprime l'interface web, `--csv` écrit les résultats exploitables et `--html` produit un rapport lisible. Palier 1 :

```
locust -f locustfile.py --host "$APP_URL" \
  --users _______ --spawn-rate _______ --run-time 3m \
  --headless --csv "palier1" --html "palier1.html"
```

Répétez avec les paliers suivants, en changeant `--users`, `--spawn-rate` et les noms de fichiers. Choisissez vos quatre valeurs d'utilisateurs de façon à ce qu'elles couvrent au moins un ordre de grandeur.

Palier	`--users`	`--spawn-rate`	Fichiers produits
1	`_______`	`_______`	`palier1_stats.csv`, `palier1.html`
2	`_______`	`_______`	`palier2_*`
3	`_______`	`_______`	`palier3_*`

4	`_______`	`_______`	`palier4_*`

Les percentiles se lisent dans les fichiers `*_stats.csv`, sur la ligne `Aggregated` : `head -1 palier1_stats.csv` donne l'en-tête, `grep "Aggregated" palier*_stats.csv` donne la ligne des quatre paliers.

Mesure 13 — débit, percentiles et taux d'échec sur les quatre paliers de charge. Reportez vos mesures. **Aucune de ces cellules ne doit être remplie avant d'avoir été lue dans un fichier CSV.**

Palier	Utilisateurs	Débit (req/s)	p50 (ms)	p95 (ms)	p99 (ms)	Taux d'échec (%)	Échecs les plus fréquents
1							
2							
3							
4							

Mesurez également la ligne de base réseau, pour pouvoir soustraire ce qui n'appartient pas à votre service : relevez la médiane sur vingt exécutions de `curl -s -o /dev/null -w "%{time_connect}\n" "$APP_URL/health"`, qui ne mesure que l'établissement de la connexion.

Mesure 14 — ligne de base réseau, à soustraire de vos percentiles.

Ligne de base	Votre valeur
Temps d'établissement de connexion, médiane sur 20 mesures	
Part que cela représente dans votre p50 du palier 1	

5.3 — Point de rupture et analyse

Le point de rupture est une définition avant d'être un chiffre. Vous ne pouvez pas le mesurer sans l'avoir défini. Posez votre critère maintenant, avant de regarder vos courbes, et tenez-vous-y.

Élément du critère	Votre choix	Justification en une ligne
Seuil de taux d'échec jugé inacceptable	`_______` %	
Seuil de p95 jugé inacceptable	`_______` fois le p95 du palier 1	
Le point de rupture est atteint quand	l'un des deux critères / les deux	

Mesure 15 — point de rupture constaté au regard du critère posé ci-dessus. Puis concluez :

Conclusion	Votre réponse
Palier où le critère est franchi pour la première fois	
Nombre d'utilisateurs simultanés supporté, borné entre deux paliers	
Débit maximal observé avant dégradation, en requêtes par seconde	
Ressource qui sature en premier, d'après les métriques de la partie 4.3	

Ce que valait `Http5xx` au palier le plus chargé	

Question 5.2 — Sur votre palier le plus chargé, comparez p50 et p99. Un écart faible et un écart important ne décrivent pas le même problème. Dites lequel des deux vous observez, et ce qu'il implique pour un praticien qui soumet une lésion en consultation : dans quelle proportion des cas subit-il le p99 ? Quatre lignes.

Question 5.3 — À partir de vos seules mesures, écrivez pour DermaScan un engagement de service en une phrase, de la forme : « jusqu'à N utilisateurs simultanés, 95 % des prédictions sont rendues en moins de X ms, avec moins de Y % d'échecs. » Les trois valeurs sont les vôtres et doivent être adossées à un palier mesuré. Un engagement pris sur un palier non mesuré ne vaut rien.

Question 5.4 — DermaScan annonce l'ouverture de deux centres supplémentaires, soit une charge doublée. Sans rien mesurer de plus, quelles sont vos **deux** options pour tenir l'engagement que vous venez d'écrire ? Nommez-les avec les termes techniques exacts, et dites laquelle vous testeriez en premier et pourquoi.

Question 5.5 — Votre API charge le modèle **une fois au démarrage du processus**, décision prise en séance 2. Qu'auriez-vous mesuré aujourd'hui si elle le rechargeait à chaque requête ? Dites sur laquelle des colonnes de la mesure 13 l'effet serait le plus visible, et pourquoi ce n'est pas le débit **aux paliers non saturés**.

Rassemblez tout ceci dans `RAPPORT_CHARGE.md`, avec en pièces jointes les quatre `*_stats.csv` et les quatre `*.html`.

Partie 6 — Gouvernance : journalisation des inférences

La gouvernance est un fil transversal du module. En séance 1, vous avez produit la fiche d'identité du conteneur. En séance 2, vous l'avez enrichie en *model card* : le modèle, ses données, ses métriques, ses limites. Cette fiche décrit un modèle **au repos**.

Depuis ce matin, votre modèle n'est plus au repos : il rend des décisions à des utilisateurs réels, et chacune de ces décisions peut être contestée. Un dermatologue qui a classé une lésion en simple surveillance parce que votre API renvoyait une probabilité sous le seuil, et dont le patient revient six mois plus tard avec un mélanome, produira une réclamation. La seule question qui comptera alors : **qu'avez-vous gardé de cette inférence ?**

Et elle se dédouble, parce que votre décision dépend d'un seuil que vous avez choisi : **quel seuil était appliqué ce jour-là ?** Un journal qui garde la décision sans garder le seuil ne permet pas de répondre.

6.1 — Que journaliser, que ne jamais journaliser

Vous avez relevé en partie 4.1 si votre API écrit ou non la charge utile complète dans ses logs. Vous allez maintenant décider ce qu'elle **devrait** écrire.

Complétez cette grille dans `JOURNALISATION_INFERENCES.md`. Les trois premières lignes sont remplies à titre d'exemple ; traitez les autres.

Donnée	Journaliser ?	Forme	Justification
Horodatage de la requête, en UTC	Toujours	Brut	Sans ordre temporel, aucune reconstitution d'incident n'est possible
Identifiant de corrélation de la requête	Toujours	Brut, généré si absent	Seul moyen de relier une ligne de log à une réclamation client
Version du modèle et digest de l'image	Toujours	Brut	Répond à « quel modèle a produit cette réponse », question n° 1 d'un audit
Identifiant du patient (`patient_id`)	`_____`	`_____`	
Identifiant de la lésion (`isic_id`)	`_____`	`_____`	
Âge et sexe (`age_approx` , `sex`)	`_____`	`_____`	
Site anatomique (`anatom_site_general`)	`_____`	`_____`	
Valeurs brutes des variables morphologiques (`tbp_lv_*`)	`_____`	`_____`	
Probabilité maligne prédite	`_____`	`_____`	
Seuil appliqué et rappel visé	`_____`	`_____`	
Décision rendue (`a_examiner` / `surveillance`)	`_____`	`_____`	
Latence d'inférence	`_____`	`_____`	

Code HTTP et message d'erreur éventuel	`_____`	`_____`	
Adresse IP source de l'appel	`_____`	`_____`	
Identité du praticien ayant déclenché l'appel	`_____`	`_____`	
Jeton d'authentification ou clé d'API	`_____`	`_____`	
Sel de pseudonymisation `INFERENCE_LOG_SALT`	`_____`	`_____`	

Question 6.1 — Le RGPD distingue les données à caractère personnel (article 4) et, parmi elles, les **données concernant la santé**, qui relèvent des catégories particulières de l'article 9 et dont le traitement est interdit par principe, sauf exception nommée. Vos variables ISIC ne tombent pas toutes dans la même case. Traitez les quatre cas séparément, en deux lignes chacun, et dites pour chacun **article 4, article 9, ou ni l'un ni l'autre** :

1. les variables morphologiques `tbp_lv_*` seules, sans aucun identifiant ;
2. les mêmes, associées à `patient_id` — sachant qu'un seul patient porte jusqu'à 9 184 lésions dans ce jeu ;
3. les mêmes, associées à `age_approx`, `sex` et `anatom_site_general`, sans `patient_id` ;
4. la **probabilité maligne prédite**, associée à un patient identifiable.

Le cas 4 est le plus instructif : une prédiction n'est pas une donnée collectée, c'est une donnée **produite** par vous. Dites si cela change quelque chose à sa qualification.

Question 6.2 — Le cas 3 porte un piège classique de la réidentification. Avec quelques variables non identifiantes prises isolément — un âge approché, un sexe, un site anatomique, une taille de lésion —, à partir de combien de variables croisées estimez-vous qu'un patient d'un centre donné redevient identifiable ? Vous ne pouvez pas le calculer aujourd'hui : nommez la mesure qu'il faudrait pour y répondre.

Question 6.3 — Votre journal fuite intégralement sur un forum public. Pour chaque ligne de votre grille marquée « Toujours », dites en trois mots ce que l'attaquant en tire. Si l'une d'elles lui donne un avantage sérieux, c'est qu'elle n'aurait pas dû être marquée « Toujours ». Traitez à part le cas de la ligne « probabilité maligne » : une fuite de prédiction est-elle moins grave qu'une fuite de diagnostic ?

6.2 — Format de ligne de journal et pseudonymisation

Écrivez dans `JOURNALISATION_INFERENCES.md` le format exact d'une ligne de journal d'inférence, au format JSON, une ligne par inférence. Le format structuré n'est pas une coquetterie : il conditionne la possibilité d'interroger les journaux autrement qu'à l'œil.

Modèle à produire : une ligne JSON par inférence, portant les champs suivants — et **rien** de ce que votre grille de 6.1 interdit.

Champ	Forme ou exemple	Ce que vous décidez
`ts`	horodatage UTC ISO 8601 à la milliseconde	Rien, c'est imposé

`request_id`	identifiant de corrélation de la requête	D'où il vient lorsque l'appelant n'en fournit pas
`model_version`	version du modèle servi	Sa source : environnement ou paquet sérialisé
`image_digest`	`sha256:...`	Digest ou tag, et pourquoi
`patient_ref`	identifiant de patient **pseudonymisé**	Jamais en clair : voir la transformation ci-dessous
`probabilite_maligne`	un nombre entre 0 et 1, par exemple `0.0412`	Le nombre de décimales conservées, et ce qu'en perdre coûterait à une analyse de dérive
`seuil_applique`	le seuil effectif au moment de l'appel	Sa source : paquet sérialisé ou variable d'environnement — et les deux ne disent pas la même chose
`decision`	`a_examiner` ou `surveillance`	Le vocabulaire retenu, repris de la séance 2
`inference_ms`	durée de l'inférence	Ce qui est chronométré, exactement
`http_status`	code retourné à l'appelant	Rien
`input_schema_version`	version du contrat d'entrée	Ce qui vous obligerait à l'incrémenter

Un champ de plus que vous n'auriez pas deviné, et c'est la séance 5 qui le réclame. La détection de dérive compare la distribution des entrées servies à celle de l'entraînement. Pour cela, elle a besoin des **covariables** de chaque inférence, pas seulement de sa sortie. Décidez donc, et justifiez : conservez-vous dans la ligne de journal un sous-ensemble des variables d'entrée — lesquelles, sous quelle forme —, ou bien rien du tout ? Les deux réponses sont défendables ; celle qui ne garde rien rend la séance 5 impossible sans un second dispositif de collecte, et vous devez dire lequel.

Pseudonymisation de `patient_id`. Le champ `patient_ref` ne doit pas contenir l'identifiant en clair. La transformation de référence, à décrire dans votre rendu : concaténer le sel lu dans l'environnement et l'identifiant, hacher le tout en SHA-256, ne conserver que les seize premiers caractères hexadécimaux.

`patient_id` n'est pas une variable du modèle : il ne figure pas parmi les quarante colonnes de `charge.json`. Il parviendrait au service hors de la charge utile, par exemple dans un en-tête `X-Patient-Ref` émis par le poste de consultation.

```python
import hashlib
import os

def pseudonymiser(patient_id: str) -> str:
    """patient_ref stable dans le temps, mais non réversible sans le sel."""
    SEL = os.environ["INFERENCE_LOG_SALT"]      # jamais en dur dans le code
    return hashlib.sha256(f"{SEL}:{patient_id}".encode("utf-8")).hexdigest()[:16]
```

Question 6.4 — Répondez aux quatre questions dans votre rendu :

1. Pourquoi le sel est-il indispensable ? Un `sha256(patient_id)` sans sel serait-il réversible, sachant que le jeu ISIC compte 1 042 patients et que les identifiants suivent un format régulier ?

2. Le résultat est **stable** : le même `patient_id` donne toujours le même `patient_ref`. En quoi est-ce nécessaire pour l'exploitation — pensez au patient qui porte plusieurs lésions —, et en quoi est-ce une faiblesse ?
3. Qui, chez DermaScan, doit pouvoir remonter de `patient_ref` à `patient_id`, et par quel moyen ?
4. Le sel est stocké en réglage d'application. Que devient l'intégralité de votre journal historique si quelqu'un fait tourner ce sel ? Dites si c'est un défaut ou une propriété recherchée.

6.3 — Durées de conservation

Une donnée conservée sans durée définie est une donnée conservée indéfiniment. Fixez vos durées et justifiez-les par un **besoin nommé**, jamais par « au cas où ».

Catégorie de trace	Durée de conservation	Besoin qui la justifie	Ce qui se passe à l'échéance
Journal d'inférence complet	`_____`		
Journal d'erreurs applicatives	`_____`		
Métriques agrégées, sans identifiant	`_____`		
Journal d'accès de la plateforme	`_____`		

Question 6.5 — Deux exigences tirent les durées dans des sens opposés : la minimisation des données et la capacité d'audit a posteriori. Nommez-les toutes les deux, puis proposez le mécanisme technique qui permet de satisfaire les deux à la fois sans choisir. Indice : il est déjà présent dans votre grille ci-dessus, à la troisième ligne.

Question 6.6 — Vous n'implémentez pas la purge aujourd'hui. Nommez l'endroit du système où elle devrait être configurée, et dites ce qui se passe si vous la configurez uniquement dans votre code applicatif alors que la plateforme conserve ses propres copies.

Nettoyage obligatoire

Un plan App Service facture à l'heure, que votre service reçoive du trafic ou non. C'est la différence de nature avec le registre de la séance 1, facturé au stockage, que vous pouviez oublier quelques jours sans dommage grave. Ici, un plan oublié pendant le week-end se voit sur le solde.

Pourquoi cette section décide de la suite du module. Votre abonnement *Azure for Students* n'est pas seulement une réserve d'argent : c'est lui qui vous donne accès à `Microsoft.ContainerRegistry` et `Microsoft.ContainerInstance`. **Un crédit épuisé ne vous laisse pas un abonnement pauvre : il le fait basculer sur l'offre *Starter*, qui interdit purement et simplement ces deux fournisseurs.** Vous n'y perdez pas la possibilité de payer, vous y perdez votre registre — donc l'artefact que les séances suivantes consomment. Épuiser son crédit en séance 3, c'est ne plus pouvoir faire la séance 4, la séance 5, ni le projet fil rouge. Le nettoyage n'est pas une bonne pratique de plus : c'est ce qui vous permet de terminer le module.

C'est en séance 3 que le risque est maximal : c'est la première séance qui fait tourner une ressource facturée à l'heure, et c'est aussi celle où un test de charge peut consommer beaucoup en peu de temps.

Ne lancez cette section qu'après avoir sauvegardé vos CSV, vos rapports HTML, vos captures d'écran et vos quatre fichiers de rendu.

Dressez l'inventaire AVANT destruction avec `az resource list --resource-group "$RG" --output table`. Lisez-le : c'est la dernière occasion de constater ce que vous avez réellement créé.

Mesure 16 — inventaire de ce qui a réellement été créé, avant destruction.

Relevé avant destruction	Votre valeur
Nombre de ressources dans le groupe	
Types de ressources listés	
Le registre ACR est-il dans **ce** groupe ou dans le groupe du registre ?	

La suppression du groupe emporte tout ce qu'il contient, y compris l'identité de l'application — mais **pas** l'attribution de rôle, qui resterait orpheline sur le registre : listez-la et supprimez-la explicitement, **avant** le groupe, tant que `$PRINCIPAL_ID` désigne encore une identité existante. Terminez par le contrôle final et le nettoyage local de l'environnement Locust.

```
az role assignment list --scope "$ACR_ID" --output table   # colonne Principal : l'appId, pas $PRINCIPAL_ID
az role assignment delete --assignee-object-id "$PRINCIPAL_ID" --scope "$ACR_ID"
az group delete --name "$RG" --yes --no-wait
# Contrôle définitif. `--no-wait` rend la main AVANT que la suppression n'aboutisse :
# `az group list` peut encore afficher le groupe pendant plusieurs minutes.
az group exists --name "$RG"   # doit finir par renvoyer false
az group list --output table
deactivate 2>/dev/null; rm -rf .venv-locust
```

Mesure 17 — contrôles de destruction, attribution de rôle orpheline comprise.

Contrôle de fin de séance	Votre observation
Le groupe `_____` apparaît-il encore ?	
Reste-t-il une attribution de rôle sur le registre ?	
Le groupe du registre est-il, lui, toujours debout ?	
`az group exists` renvoie	
Crédit restant, relevé dans `Cost Management`	
Écart avec le relevé de la séance précédente	

Question 7.1 — Vous devez conserver le registre ACR pour la séance 4 et le projet fil rouge, mais détruire tout le reste. Qu'est-ce qui, dans la façon dont vous avez organisé vos groupes de ressources, rend cette opération sûre en une seule commande ? Qu'auriez-vous dû faire de plus si le registre avait été dans le même groupe que l'application web ?

Avant de quitter la salle — extinction

Toute ressource active est une ressource que vous payez. Vous payez la *réservation*, pas l'usage : un service que personne n'appelle, la nuit et le week-end, coûte exactement le même prix qu'un service sollicité. C'est cette règle, et elle seule, qui décide si vous finirez le module avec du crédit.

Trois gestes, dans cet ordre. Aucun n'est facultatif, et ils se font **avant** de fermer votre session, pas « plus tard depuis chez vous » — un oubli du vendredi se paie jusqu'au lundi.

1. **Côté Azure. C'est la séance où l'oubli coûte le plus cher.** Un plan App Service facture à l'heure, trafic ou non. Le groupe de la séance est détruit, l'attribution de rôle orpheline supprimée, et `az group exists` doit finir par renvoyer `false`. Seul le groupe du registre survit.
2. **Côté poste — arrêtez ce qui tourne encore.** Conteneurs de la séance supprimés, Docker Desktop **quitté** (un démon laissé actif relance vos conteneurs au prochain démarrage et consomme mémoire et batterie), environnement virtuel désactivé.
3. **Éteignez votre machine.** Éteinte, pas en veille.

```
docker ps -a                # plus aucun conteneur de la séance
az group list --output table # le groupe de la séance a disparu, celui du registre demeure
```

Contrôle d'extinction	Fait
`az group exists` sur le groupe de la séance renvoie `false`	
Aucun conteneur de la séance dans `docker ps -a`	
Docker Desktop quitté	
Machine éteinte	

Livrable attendu

Un dossier `s3-<votre-nom>/` déposé sur le dépôt de classe avant la fin de la séance :

Fichier	Contenu
`FAISABILITE.md`	Les mesures 1 et 2, et les questions 0.1 à 1.2
`DEPLOIEMENT.md`	Les mesures 3 à 8 et 16-17, les questions 2.1 à 3.4 et 7.1, **dont le seuil relevé de S2 et la valeur abaissée testée en 3.3**
`JOURNAL_INCIDENT.md`	Les mesures 9 à 12 et les questions 4.1 et 4.2, au format compte rendu d'incident : symptôme, ligne de log fautive recopiée, cause, correction, délai de retour au service
`locustfile.py`	Votre scénario, blancs remplis, poids justifiés
`palier[1-4]_stats.csv`, `palier[1-4].html`	Les sorties brutes de Locust, non retouchées
`RAPPORT_CHARGE.md`	Les mesures 13 à 15, le critère de rupture posé **avant** mesure, et les questions 5.1 à 5.5
`JOURNALISATION_INFERENCES.md`	La grille de 6.1, le format de ligne de 6.2, la pseudonymisation, les durées de 6.3, et les questions 6.1 à 6.6
`charge.json`	La charge utile réelle reprise de S2, celle qui a servi à tous vos appels
`captures/`	Au minimum : la réponse `200` de `/health` sur l'URL publique, une réponse `/predict` complète avec sa `decision` et son `seuil`, la ligne de log de l'incident, un rapport HTML de Locust

Ce qui n'est pas demandé et ne sera pas noté : un nouveau modèle, un réseau de neurones, une optimisation d'inférence, un manifeste Kubernetes. Ces éléments sont l'objet des séances 4, 5 et du projet fil rouge. Les produire aujourd'hui ne rapporte aucun point.

Annexe A — Dépannage

Symptôme	Cause la plus fréquente	Action
`Operation cannot be completed without additional quota`	Quota nul sur App Service pour cet abonnement	Basculer sur l'annexe B, section B.1
`RequestDisallowedByPolicy`	Région hors de la liste autorisée par la politique	Reprendre la sonde 1 de la partie 1.2 et choisir une région autorisée
`MissingSubscriptionRegistration`	Fournisseur de ressources non enregistré	`az provider register --namespace <nom> --wait`
`Website with given name already exists`	Nom d'application non unique sur Azure	Changer de suffixe dans `APP_NAME`
Page « Application Error » ou « 503 Service Unavailable » persistante	L'image n'a pas pu être tirée	Vérifier le rôle `AcrPull` et son étendue, puis `az webapp restart`
`ImagePullUnauthorizedFailure` (`Image pull failed with forbidden or unauthorized`) dans les logs	Rôle non propagé, ou `acrUseManagedIdentityCreds` non activé	Attendre 2 min, revérifier `az webapp config show`
`Container did not respond to startup probe on port …` suivi de `Port mismatch detected`	`WEBSITES_PORT` incohérent avec le port réel d'écoute	Aligner `WEBSITES_PORT` , `PORT` et le `EXPOSE` de l'image
`exec format error`	Image construite en `arm64`	Reconstruire avec `--platform linux/amd64` et republier
`az webapp log tail` reste vide, ou aucune ligne de Gunicorn dans l'archive	Journalisation non activée : sans elle, seuls les journaux de la plateforme sont conservés	Rejouer `az webapp log config` puis regénérer du trafic
`Log stream interrupted` dans `az webapp log tail`	Le conteneur a été recyclé, par exemple après un `appsettings set`	Relancer `az webapp log tail`
Locust : 100 % d'échecs `ConnectionError`	`--host` sans le schéma `https://`	Vérifier `echo "$APP_URL"`
Locust : latences très supérieures aux `curl` manuels	Poste client saturé, ou Wi-Fi partagé	Réduire `--users` , passer en filaire, et le **noter dans le rapport**
`--deployment-container-image-name is deprecated`	Ancienne syntaxe de CLI	Utiliser `--container-image-name`
Sous PowerShell : `export` inconnu	Syntaxe shell	`$env:NOM = "valeur"` , et `$env:NOM` pour la relire

Annexe B — Plan de repli sans App Service

Cette annexe est **opérationnelle** : les commandes sont complètes. Si la partie 1 vous y a renvoyé, vous y passez et vous reprenez ensuite les parties 3 à 6 de l'énoncé principal, avec les correspondances données en B.4. **Aucun point n'est perdu** : vous ne contrôlez pas les quotas de votre abonnement.

B.1 — Cible B : Azure Container Instances

ACI exécute un conteneur sans plan de service ni machine à gérer. C'est le repli le plus proche de l'original.

Étape 1 — donner à ACI le droit de tirer l'image. ACI ne peut pas utiliser l'identité managée d'App Service. Créez un principal de service limité au rôle de lecture sur votre registre.

Le principal créé n'a qu'un seul droit : tirer des images de **ce** registre. Relevez `appId` et `password` dans la sortie et reportez-les dans `SP_ID` et `SP_PASSWORD` : ils ne seront plus réaffichés.

```
export LOGIN_SERVER=$(az acr show --name "$ACR_NAME" --query loginServer --output tsv)
export IMAGE_FULL="$LOGIN_SERVER/$IMAGE_REPO:$IMAGE_TAG"   # comme en 2.1
export ACR_ID=$(az acr show --name "$ACR_NAME" --query id --output tsv)
az ad sp create-for-rbac --name "sp-acrpull-$SUFFIX" --role AcrPull \
  --scopes "$ACR_ID" --output json
export SP_ID="_______"
export SP_PASSWORD="_______"
```

Si votre compte ne vous autorise pas à créer un principal de service dans Entra ID — c'est fréquent sur un tenant scolaire —, utilisez les identifiants administrateur du registre (`az acr update --name "$ACR_NAME" --admin-enabled true` , puis `az acr credential show` pour lire `username` et `passwords[0].value` dans `SP_ID` et `SP_PASSWORD`), **en le mentionnant explicitement dans** `FAISABILITE.md` comme une dégradation subie et non choisie.

Étape 2 — créer le conteneur.

Cette commande est la référence de l'annexe : les étapes 4 et 5 n'en changent que quelques options. L'URL publique se reconstruit ensuite depuis le FQDN, de la forme `<dns-label>.<region>.azurecontainer.io` .

```
export DNS_LABEL="dermascan-$SUFFIX"
export INFERENCE_SALT=$(openssl rand -hex 16)
az container create --resource-group "$RG" --name "aci-dermascan-$SUFFIX" \
  --image "$IMAGE_FULL" --registry-login-server "$LOGIN_SERVER" \
  --registry-username "$SP_ID" --registry-password "$SP_PASSWORD" \
  --dns-name-label "$DNS_LABEL" --ports 8000 --cpu 1 --memory 1.5 \
  --os-type Linux --restart-policy OnFailure \
  --environment-variables PORT=8000 MODEL_VERSION="_______" \
      LOG_LEVEL="_______" ENVIRONMENT=recette SEUIL="_______" \
  --command-line "gunicorn -b 0.0.0.0:8000 --workers 2 --threads 4 app:app" \
  --secure-environment-variables INFERENCE_LOG_SALT="$INFERENCE_SALT"
export APP_URL="http://$(az container show --resource-group "$RG" \
  --name "aci-dermascan-$SUFFIX" --query ipAddress.fqdn --output tsv):8000"
```

Sans frontal, ACI expose directement Gunicorn. Les connexions inactives que la plateforme ouvre vers le conteneur bloquent un worker `sync` unique — celui que lance l'image de S2 : on a mesuré 5 à 20 s par requête, contre 0,08 à 0,14 s avec `--workers 2 --threads 4`. `--command-line` remplace le `CMD` de l'image sans la reconstruire. Adaptez `app:app` au module de votre image.

Étape 3 — observabilité. Trois sources, toutes sur le même conteneur `aci-dermascan-$SUFFIX` du groupe `$RG`.

Source	Commande	Ce qu'on y lit
État de l'instance	`az container show --query "{etat:instanceView.state, redemarrages:containers[0].instanceView.restartCount}"`	L'état courant et le nombre de redémarrages
Logs applicatifs	`az container logs --follow`	Équivalent direct de `az webapp log tail` ; bloquant comme lui : Ctrl+C pour rendre la main
Événements de plateforme	`az container show --query "containers[0].instanceView.events"`	C'est **ici**, et nulle part ailleurs, que se lit un échec de pull, et un redémarrage décidé par la plateforme (`Liveness probe failed:` puis `Killing container … (platform initiated)`)

Dans les événements, `Pulling` et `Pulled` nomment l'image par le digest du manifeste `amd64` (`dermascan-api@sha256:…`), pas par celui de l'index relevé en 2.1 : l'index ne fait que pointer vers ce manifeste, ce n'est pas une autre image.

La sonde de santé d'ACI (`livenessProbe`, voir B.4) ne se déclare pas en ligne de commande : il faut décrire tout le groupe de conteneurs dans un fichier YAML et le passer à `az container create --resource-group "$RG" --file aci.yaml`. Extrait vérifié le 23/10/2026 ; remplacez les valeurs entre chevrons et complétez `environmentVariables` avec les autres variables de l'étape 2 (`PORT`, `MODEL_VERSION`, `LOG_LEVEL`, `ENVIRONMENT`).

```
apiVersion: '2023-05-01'
location: <région>
name: aci-dermascan-<suffixe>
type: Microsoft.ContainerInstance/containerGroups
properties:
  osType: Linux
  restartPolicy: OnFailure
  imageRegistryCredentials:
  - {server: <loginServer>, username: <SP_ID>, password: <SP_PASSWORD>}
  ipAddress: {type: Public, dnsNameLabel: dermascan-<suffixe>, ports: [{protocol: TCP,
port: 8000}]}
  containers:
  - name: aci-dermascan-<suffixe>
    properties:
      image: <IMAGE_FULL>
      command: [gunicorn, -b, 0.0.0.0:8000, --workers, '2', --threads, '4', app:app]
      ports: [{port: 8000}]
      resources: {requests: {cpu: 1, memoryInGB: 1.5}}
      environmentVariables:
      - {name: SEUIL, value: '0.288459'}
      - {name: INFERENCE_LOG_SALT, secureValue: '<sel>'}
      livenessProbe:
```

```
        httpGet: {path: /health, port: 8000, scheme: http}
        initialDelaySeconds: 20
        periodSeconds: 10
        timeoutSeconds: 3
        failureThreshold: 3
```

Ce fichier contient le mot de passe du registre et le sel : ne le versionnez pas. Avec un worker `sync` unique, cette sonde tue le conteneur toutes les minutes environ — trois redémarrages en quatre minutes le 23/10/2026, avec `Liveness probe failed:` puis `Killing container … (platform initiated)` dans les événements, et rien dans les logs. D'où la ligne `command`, équivalent YAML du `--command-line` de l'étape 2 : ne la retirez pas. Elle a été ajoutée après le test (le fichier vérifié ne l'avait pas, d'où les redémarrages) ; si elle est refusée, appliquez au moins la correction de l'étape 2 avant d'ajouter une sonde.

Étapes 4 et 5 — la panne provoquée, puis la montée en charge. Ces deux étapes rejouent la commande de l'étape 2 en n'en changeant que trois options. Ne retapez pas tout : reprenez la commande de référence et substituez les valeurs de ce tableau. Le levier `WEBSITES_PORT` n'existe pas sur ACI ; l'équivalent exact de la panne de 4.2 est un conteneur qui publie un port sur lequel l'application n'écoute pas.

Étape	`--name`	`--dns-name-label`	`--ports`	Préalable
4 — panne provoquée	`aci-dermascan-$SUFFIX`	`$DNS_LABEL-ko`	`9999`	`az container delete --yes` sur l'instance saine
5 — deuxième instance	`aci-dermascan-$SUFFIX-2`	`$DNS_LABEL-2`	`8000`	aucun : la première instance reste en service

Relevez pour l'étape 4 la même grille de diagnostic qu'en 4.2 (**mesure 10**), adaptée : relevez le code ou l'erreur vue par le client — réinitialisation de connexion (`curl: (56) Recv failure: Connection reset by peer` en environ 2 s, observé le 23/10/2026) ou délai d'attente, ACI n'ayant pas de frontal pour répondre `503` à sa place —, et constatez que ni les logs ni les événements ne nomment la cause : ACI ne sonde pas le port. Ce silence **est** le relevé.

ACI ne met pas à l'échelle automatiquement : c'est pourquoi l'étape 5 crée une seconde instance à la main. Lancez Locust sur une instance, relevez le point de rupture, puis sur l'autre en parallèle. Le concept mesuré est le même, la mise à l'échelle est simplement manuelle. **Dites-le dans votre rapport** : c'est précisément ce qu'un service managé automatise.

Nettoyage ACI : `az container delete` sur les deux instances (`--yes`), puis, si vous avez créé un principal de service, `az role assignment delete --assignee "$SP_ID" --scope "$ACR_ID"` et `az ad app delete --id "$SP_ID"` (supprimer l'application supprime aussi son principal de service ; `az ad sp delete` seul laisse l'application, et l'attribution `AcrPull` resterait orpheline sur le registre conservé jusqu'en S4), `az acr update --name "$ACR_NAME" --admin-enabled false` si vous l'aviez activé, et enfin `az group delete --name "$RG" --yes --no-wait`.

B.2 — Cible B bis : machine virtuelle Azure avec Docker

Si ACI est également refusé — c'est arrivé en 25-26 —, les machines virtuelles restent disponibles. Vous perdez le caractère managé, vous conservez tout le reste, et vous y gagnez une URL publique réelle.

La logique. Quatre gestes côté Azure : une clé SSH dédiée au TP, une machine Ubuntu de la plus petite taille facturée, l'ouverture du port applicatif dans le groupe de sécurité réseau, et le relevé de l'IP publique qui devient votre `APP_URL`. Puis trois gestes sur la machine, par SSH : installation de `docker.io`, authentification au registre avec le principal de service de B.1, et lancement du conteneur avec les mêmes variables d'environnement que sur les autres cibles. `sudo docker logs -f dermascan-api` remplace alors `az webapp log tail`.

```
ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa_dermascan -N "" -C "dermascan-s3"
az vm create --resource-group "$RG" --name "vm-dermascan-$SUFFIX" \
  --image Ubuntu2204 --size Standard_B1s --admin-username azureuser \
  --ssh-key-values "@$HOME/.ssh/id_rsa_dermascan.pub" --nsg-rule SSH
az vm open-port --resource-group "$RG" --name "vm-dermascan-$SUFFIX" \
  --port 8000 --priority 1001
export VM_IP=$(az vm show --resource-group "$RG" --name "vm-dermascan-$SUFFIX" \
  --show-details --query publicIps --output tsv)
export APP_URL="http://$VM_IP:8000"
```

Extrait : le lancement du conteneur, seul point où le mot de passe du registre transite. Il passe par `--password-stdin` et non par `-p`, pour ne pas apparaître dans la table des processus de la machine distante.

```
ssh -i ~/.ssh/id_rsa_dermascan azureuser@"$VM_IP" \
  "echo '$SP_PASSWORD' | sudo docker login '$LOGIN_SERVER' -u '$SP_ID' --password-stdin \
   && sudo docker run -d --name dermascan-api --restart unless-stopped \
      -p 8000:8000 -e PORT=8000 -e LOG_LEVEL=info -e ENVIRONMENT=recette \
      -e MODEL_VERSION='$MODEL_VERSION' -e SEUIL='$SEUIL' \
      -e INFERENCE_LOG_SALT='$INFERENCE_SALT' '$IMAGE_FULL'"
```

`MODEL_VERSION` et `SEUIL` sont à exporter au préalable avec les valeurs de 3.1, et `INFERENCE_SALT` comme en B.1. Limite à déclarer dans `FAISABILITE.md` : le mot de passe passe par `--password-stdin`, mais le sel, lui, passe en `-e` sur la ligne de commande distante ; il reste lisible par `docker inspect` sur la machine, et dans la table des processus le temps du lancement.

B.3 — Cible C : exécution locale instrumentée

Dernier repli, à n'utiliser que si A et B ont échoué. Les concepts de variables d'environnement, de secrets, de logs centralisés et de montée en charge sont tous préservés ; seule l'exposition publique est perdue. **Elle vaut les mêmes points**, et vous le documentez.

Créez `compose.yaml` et `nginx.conf`. Trois répliques de votre image derrière un répartiteur : c'est la mise à l'échelle horizontale, en miniature.

Ce que `compose.yaml` doit déclarer.

Service	Éléments attendus

`api`	Votre image, via `${IMAGE_FULL}` ; un bloc `environment` portant `PORT` sous la forme `${PORT:-8000}` — surchargeable depuis le shell, c'est le levier de la panne —, `MODEL_VERSION`, `LOG_LEVEL`, `ENVIRONMENT` et le sel `${INFERENCE_SALT}` ; `expose` du port applicatif, sans publication vers l'hôte ; `deploy.replicas` à trois ; un `healthcheck` qui interroge `/health` (intervalle 10 s, délai 3 s, trois tentatives, démarrage toléré 20 s) — ni `curl` ni `wget` n'existent dans l'image (base Wolfi de la séance 1) : la sonde passe par `python3 -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"` ; un `logging` `json-file` plafonné par `max-size` et `max-file`
`passerelle`	`nginx:1.27-alpine` ; publication `8080:80` ; montage **en lecture seule** de `nginx.conf` sur `/etc/nginx/conf.d/default.conf` ; `depends_on` sur `api`

Les deux seules valeurs à décider sont `MODEL_VERSION` et `LOG_LEVEL` ; le sel et l'image viennent de l'environnement, jamais du fichier.

Ce que `nginx.conf` doit déclarer. Un `resolver 127.0.0.11` — le DNS interne de Docker, qui résout `api` vers les adresses de toutes les répliques —, un `server` en écoute sur le port 80, un `location /` qui relaie vers `http://api:8000` en propageant un identifiant de requête, et surtout un format de journal d'accès portant la latence amont : c'est **votre** source de mesure. Piège : un bloc `upstream` résout le nom **une seule fois**, au démarrage de nginx ; après un `--scale`, la passerelle ignorerait les nouvelles répliques et le point de rupture ne bougerait pas. Placez donc l'adresse dans une variable (`set $amont http://api:8000;` puis `proxy_pass $amont;`) : nginx la résout alors à nouveau à chaque expiration du `resolver`.

Extrait : le format de journal, seul point où une erreur vous priverait de toute mesure.

```
log_format charge '$remote_addr $request $status '
                  'amont=$upstream_response_time total=$request_time';
access_log /var/log/nginx/access.log charge;
```

Le sel et l'image viennent de l'environnement, jamais du fichier compose. `docker compose logs -f` centralise les trois répliques en un seul flux : c'est exactement ce que `az webapp log tail` fait pour vous sur la plateforme. L'équivalent de la panne de 4.2 consiste à faire écouter l'application sur un port que nginx ne route pas — `PORT=9999 docker compose up -d --force-recreate api` —, puis à comparer le code vu du client et les 40 dernières lignes de la passerelle.

```
export INFERENCE_SALT=$(openssl rand -hex 16)
export LOGIN_SERVER=$(az acr show --name "$ACR_NAME" --query loginServer --output tsv)
export IMAGE_FULL="$LOGIN_SERVER/$IMAGE_REPO:$IMAGE_TAG"
docker compose up -d
export APP_URL="http://localhost:8080"
curl -s "$APP_URL/health"
docker compose logs -f
```

Pour la partie 5, lancez Locust sur `http://localhost:8080` avec `--scale api=1` puis `--scale api=3`, à charge et à durée identiques, et relevez pour chaque configuration le même tableau de paliers. Le point de rupture doit se déplacer. Sur un Mac Apple Silicon, l'image `amd64` tourne sous émulation : c'est une limite de mesure, à déclarer dans votre rapport.

```
docker compose up -d --scale api=1
locust -f locustfile.py --host "$APP_URL" --users 20 --spawn-rate 4 \
  --run-time 3m --headless --csv "local_1replique"
```

Ce que ce repli vous fait gagner en plus. Vous mesurez l'effet de la mise à l'échelle horizontale sur **votre propre** point de rupture, ce que la cible A ne vous permet pas de faire à moindre coût. Exploitez-le : c'est le meilleur contenu de rapport de la séance.

Nettoyage : `docker compose down -v` puis `az group delete --name "$RG" --yes --no-wait`.

B.4 — Correspondance des commandes entre les trois cibles

Geste	A — App Service	B — ACI	C — Local
Déployer	`az webapp create --container-image-name`	`az container create --image`	`docker compose up -d`
Variables d'environnement	`az webapp config appsettings set`	`--environment-variables`	bloc `environment`
Secret	réglage d'application	`--secure-environment-variables`	variable de shell injectée
Port	`WEBSITES_PORT`	`--ports`	`expose` + `ports`
Sonde de santé	`siteConfig.healthCheckPath`	sonde `livenessProbe` dans un fichier YAML (`az container create --file` , modèle en B.1)	bloc `healthcheck`
Logs en direct	`az webapp log tail`	`az container logs --follow` (bloquant : Ctrl+C pour rendre la main)	`docker compose logs -f`
Redémarrage	`az webapp restart`	`az container restart`	`docker compose restart`
Mise à l'échelle	`az appservice plan update --number-of-workers`	création manuelle d'instances	`--scale api=N`
Détruire	`az group delete`	`az container delete`	`docker compose down -v`

Annexe C — Ce que vous emportez vers la séance 4

Acquis de S3	Usage en S4
La séquence de déploiement complète, de l'image à l'URL	**Devient une routine** : en S4, vous la rejouez sans énoncé, en 20 minutes, pour le modèle de classification d'images
L'identité managée et le rôle `AcrPull` sur le registre	Rejoués à l'identique ; vous ne redécouvrirez pas le sujet
Le réflexe « vérifier la faisabilité avant d'investir »	Appliqué au dimensionnement : un CNN n'a pas les mêmes besoins en mémoire qu'une régression logistique sur quarante variables
Votre point de rupture mesuré sur un modèle léger	**Sert de référence** : en S4, vous mesurerez le même point de rupture sur un CNN et l'écart sera le sujet de la séance
`locustfile.py`	Réutilisé, seule la charge utile change : des quarante variables tabulaires vers une vignette encodée
La grille de journalisation des inférences	Étendue en S4 à l'**image** : une vignette dermoscopique est une donnée de santé qui ne se pseudonymise pas par hachage, et c'est tout le problème
Le champ de covariables de votre ligne de journal	**Consommé en séance 5** : c'est la distribution de référence contre laquelle la dérive se mesure. Un journal sans covariables rend la séance 5 aveugle
Le réflexe de nettoyage	Appliqué à chaque fin de séance jusqu'à la soutenance

Annexe D — Image de secours

Si votre image de séance 2 est introuvable ou défectueuse, une image de référence conforme au contrat d'interface est mise à disposition par le formateur en début de séance. Poussez-la dans **votre** registre avant de commencer :

```
az acr login --name "$ACR_NAME"            # ACR_NAME : bloc de la partie 1.1
export LOGIN_SERVER=$(az acr show --name "$ACR_NAME" --query loginServer --output tsv)
docker pull "<image-de-secours-communiquée-par-le-formateur>"
docker tag "<image-de-secours>" "$LOGIN_SERVER/dermascan-api:1.0.0-secours"
docker push "$LOGIN_SERVER/dermascan-api:1.0.0-secours"
```

L'image de secours est accompagnée de son `charge.json`, puisque les colonnes attendues sont celles du modèle de référence et non celles du vôtre. **Il porte quarante colonnes et aucune valeur manquante** ; contrôlez-le comme le vôtre, avec la commande du prérequis 6. Mentionnez ce recours dans `DEPLOIEMENT.md`.

Rattrapez votre image de S2 avant la séance 4, où elle ne sera plus fournie.